
基于 Pygame 的 2D 西游记 RPG 小游戏

概要设计文档

【课程名称】：软件项目开发与实践

【项目名称】：基于 Pygame 的 2D 西游记 RPG 小游戏

【文档版本】：V1.0

【编写日期】：2026 年 6 月 24 日

【编写人员】：本人

1 引言

1.1 编写目的

本文档是《基于 Pygame 的 2D 西游记 RPG 小游戏》的概要设计文档，旨在描述系统的总体架构、模块划分、接口设计、数据结构和算法设计，为详细设计和编码实现提供指导。

本文档适用于以下读者：

- 开发人员：理解系统架构和模块职责
- 测试人员：了解系统结构以设计测试用例
- 课程教师：了解项目的设计方案

1.2 项目背景

本项目为软件项目开发课程实训项目，以 Pygame 为开发框架，实现一款以西游记“祸起观音院”为故事背景的 2D RPG 游戏。玩家扮演孙悟空，需要在村庄中与 NPC 交互获取线索和属性加成，最终前往寺庙挑战牛怪。

1.3 术语与缩写

表 1: 术语与缩写

术语/缩写	说明
RPG	Role-Playing Game, 角色扮演游戏
TMX	Tiled Map XML, Tiled 地图编辑器格式
HP	Hit Points, 生命值/血量
ATK	Attack, 攻击力
WASD	游戏操控键位（上左下右）
HUD	Heads-Up Display, 平视显示器（游戏界面信息）
FPS	Frames Per Second, 每秒帧数
Pygame	Python 游戏开发库
pytmx	Python TMX 地图解析库
OOP	面向对象编程
AI	Artificial Intelligence, 人工智能

1.4 参考资料

- Pygame 官方文档: <https://www.pygame.org/docs/>
- pytmx 库文档: <https://pytmx.readthedocs.io/>
- Tiled Map Editor: <https://www.mapeditor.org/>
- 需求规格说明文档

2 系统总体设计

2.1 系统架构设计

系统采用分层模块化设计，分为核心层、角色层、系统层、场景层和工具层，共 5 层架构。

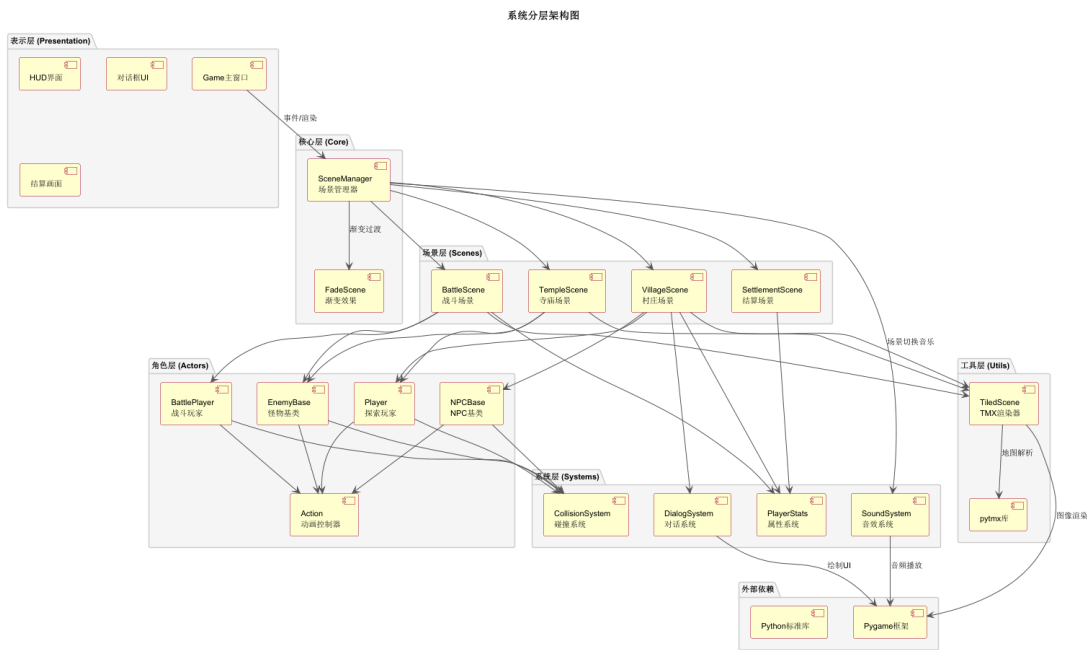


图 1: 系统分层架构图

各层职责说明:

表 2: 系统分层职责

层级	模块	职责
核心层	Game, SceneManager	游戏初始化、主循环、场景调度、渐变过渡
角色层	Player, NPC, Enemy	角色行为、动画控制、碰撞检测、AI 逻辑
系统层	Dialog, Stats, Sound, Collision	对话显示、属性管理、音效播放、碰撞检测
场景层	Village, Temple, Battle, Settlement	场景逻辑、事件处理、画面渲染
工具层	TiledScene, FadeScene	TMX 地图加载渲染、渐变效果实现

2.2 模块划分与职责

系统共分为以下主要模块:

- 1. 核心模块: Game 主类、SceneManager 场景管理器

-
- 2. 角色模块: ActorBase 基类、Player、BattlePlayer、NPCBase、EnemyBase
 - 3. 系统模块: DialogSystem、PlayerStats、SoundSystem、CollisionSystem
 - 4. 场景模块: VillageScene、TempleScene、BattleScene、SettlementScene
 - 5. 工具模块: TiledScene、FadeScene

2.3 技术选型与依据

表 3: 技术选型

技术	版本	选型依据
Python	3.0+	课程要求，语法简洁，生态丰富
Pygame	2.0+	成熟的 2D 游戏开发框架，支持精灵、碰撞、音效
pytmx	3.0+	支持 Tiled 地图编辑器格式，便于地图设计
Tiled	最新版	可视化地图编辑器，支持对象层定义出生点

3 接口设计

3.1 用户接口设计

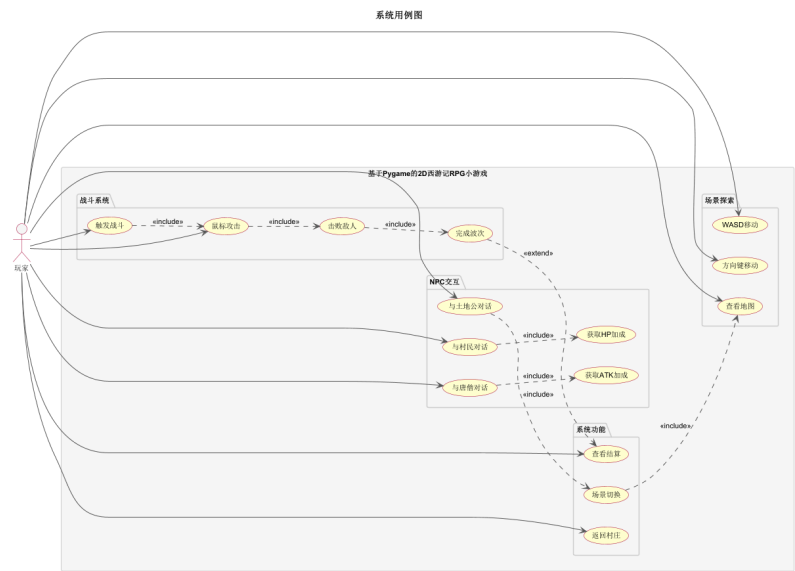


图 2: 系统用例图

用户界面元素说明:

表 4: 用户界面元素

界面元素	位置	说明
游戏窗口	屏幕中央	800×600 像素，标题“基于 Pygame 的 2D 西游记 RPG 小游戏”
HUD	屏幕下方	血条、攻击力、波次信息（战斗中）
对话框	屏幕底部	半透明黑色背景，双行文字
提示信息	角色头顶	“按空格键发起对话/战斗”
结算画面	屏幕中央	战斗结果、属性信息

3.2 内部接口设计

模块间调用关系:

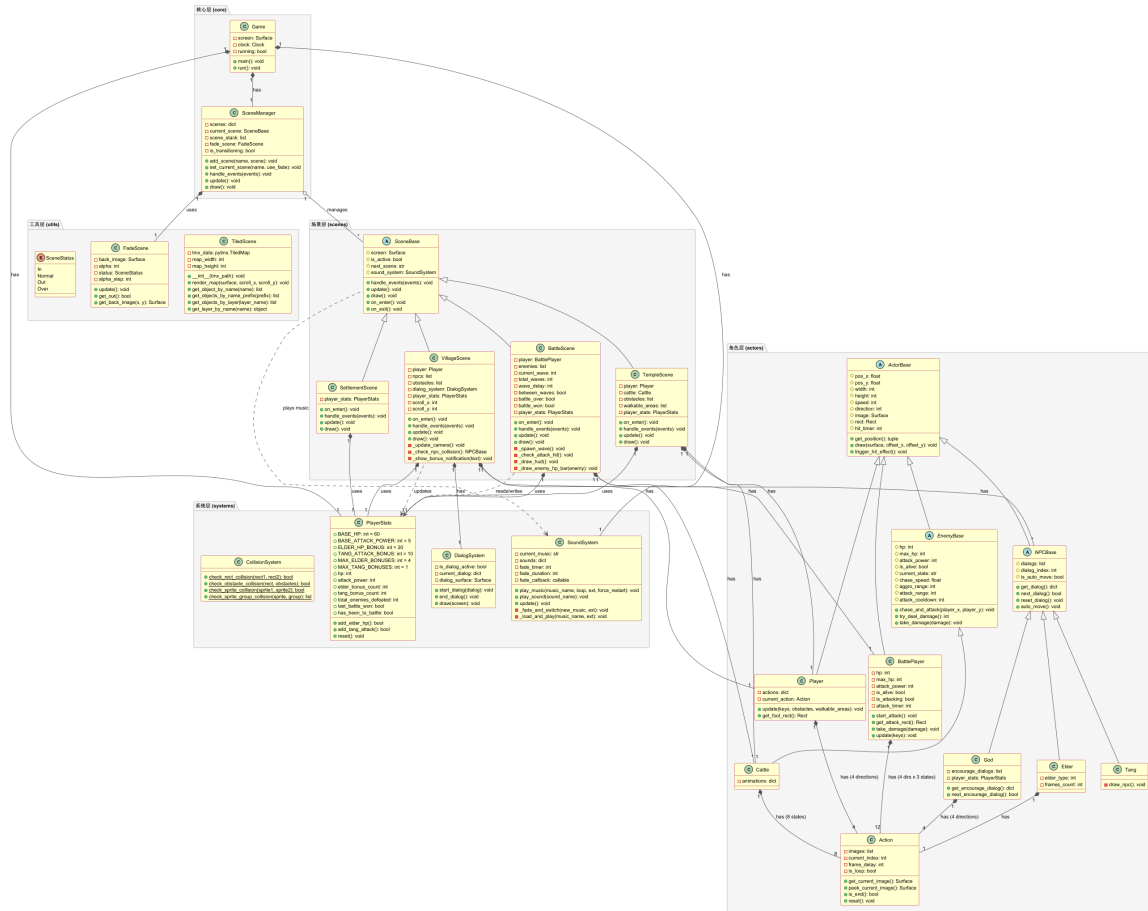


图 3: 核心类关系图

核心接口说明:

表 5: 核心接口说明

接口名称	所属模块	功能说明
set_current_scene()	SceneManager	切换当前场景，支持渐变过渡
handle_events()	SceneBase	处理用户输入事件
update()	SceneBase	更新场景状态
draw()	SceneBase	渲染场景画面
on_enter()	SceneBase	场景进入时初始化
on_exit()	SceneBase	场景退出时清理
start_attack()	BattlePlayer	开始攻击动画
get_attack_rect()	BattlePlayer	获取攻击范围
take_damage()	ActorBase	接受伤害处理

续下页

续表		
接口名称	所属模块	功能说明
chase_and_attack()	EnemyBase	敌人追击 AI 逻辑
play_music()	SoundSystem	播放背景音乐，支持渐弱切换
add_elder_hp()	PlayerStats	增加血量属性
add_tang_attack()	PlayerStats	增加攻击力属性

4 数据结构设计

4.1 核心数据类设计

4.1.1 PlayerStats 属性系统

PlayerStats 是玩家属性的持久化存储类，采用单例模式，通过引用传递给所有场景。

Listing 1: PlayerStats 类定义

```
1 class PlayerStats:
2     # 可调节的配置值
3     BASE_HP = 60           # 初始血量
4     BASE_ATTACK_POWER = 5  # 初始攻击力
5     ELDER_HP_BONUS = 20    # 每次与村民交互增加血量
6     TANG_ATTACK_BONUS = 10 # 每次与唐僧交互增加攻击力
7     MAX_ELDER_BONUSES = 4  # 最多4个村民
8     MAX_TANG_BONUSES = 1   # 1个唐僧
9
10    def __init__(self):
11        self.hp = self.BASE_HP
12        self.attack_power = self.BASE_ATTACK_POWER
13        self.elder_bonus_count = 0
14        self.tang_bonus_count = 0
15        self.total_enemies_defeated = 0
16        self.last_battle_won = False
17        self.has_been_to_battle = False
```

4.1.2 ActorBase 角色基类

Listing 2: ActorBase 类定义

```
1 class ActorBase(pygame.sprite.Sprite):
2     DOWN = 0
3     LEFT = 1
4     UP = 2
5     RIGHT = 3
6
7     def __init__(self, x, y, width, height, speed):
8         self.pos_x = x
9         self.pos_y = y
10        self.width = width
11        self.height = height
12        self.speed = speed
13        self.direction = self.DOWN
14        self.image = None
15        self.rect = None
16        self.hit_timer = 0
17        self.hit_duration = 10
18        self.original_image = None
```

4.1.3 BattlePlayer 战斗玩家

Listing 3: BattlePlayer 类定义

```
1 class BattlePlayer(ActorBase):
2     def __init__(self, x, y):
3         super().__init__(x, y, 90, 126, 4)
4         self.hp = 60
5         self.max_hp = 60
6         self.attack_power = 5
7         self.is_alive = True
8         self.is_attacking = False
9         self.attack_timer = 0
10        self.attack_duration = 20
```

4.1.4 EnemyBase 敌人基类

Listing 4: EnemyBase 类定义

```
1 class EnemyBase(ActorBase):
```

```
2  def __init__(self, x, y, width, height, hp, attack_power):
3      super().__init__(x, y, width, height, 1.5)
4      self.hp = hp
5      self.max_hp = hp
6      self.attack_power = attack_power
7      self.is_alive = True
8      self.current_state = 'station'
9      self.aggro_range = 300
10     self.attack_range = 40
11     self.attack_cooldown = 60
```

4.2 数据持久化设计

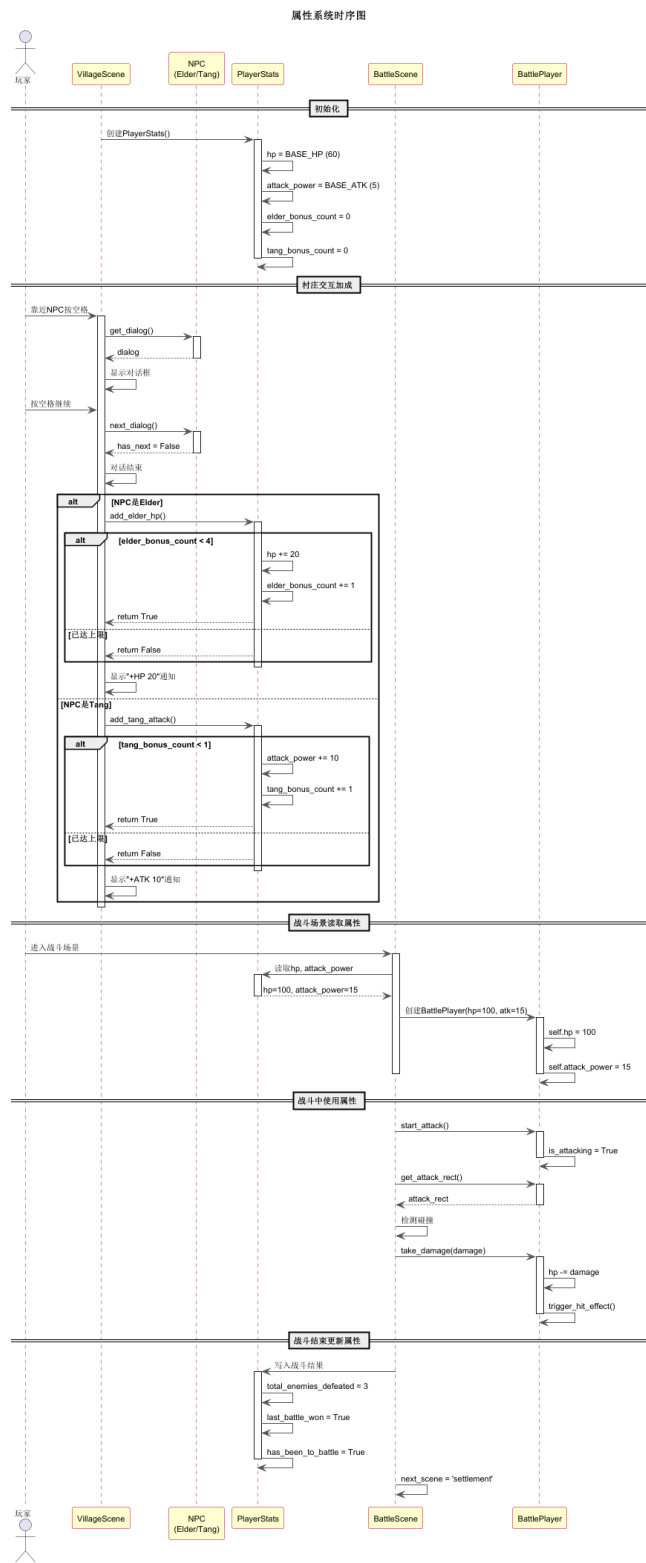


图 4: 属性系统时序图

数据持久化采用内存引用传递方式：

- 1. 创建阶段：Game 主类创建 PlayerStats 单例
- 2. 传递阶段：通过构造函数传递给各场景
- 3. 更新阶段：VillageScene 更新属性（NPC 交互加成）
- 4. 读取阶段：BattleScene 读取属性（创建 BattlePlayer）
- 5. 回写阶段：BattleScene 写回战斗结果

4.3 资源文件结构

表 6: 资源文件清单

类型	路径	说明
玩家精灵 (探索)	resource/img/swk2/	孙悟空 128 帧 PNG
玩家精灵 (战斗)	resource/img/swk/	孙悟空 16 帧 TGA
NPC 精灵	resource/img/elder/	村民 TGA 格式
怪物精灵	resource/img/cattle/	牛怪 TGA 格式
土地公	resource/img/god/	土地公 4 方向 40 帧
地图文件	resource/tmx/village1.tmx	村庄地图
地图文件	resource/tmx/temple.tmx	寺庙地图
地图文件	resource/tmx/test.tmx	战斗地图（暂时）
字体文件	resource/font/newfont.TTF	中文字体
背景音乐	resource/sound/bgm.mp3	村庄/寺庙音乐
战斗音乐	resource/sound/fight.mp3	战斗音乐

5 模块详细设计

5.1 核心层模块

5.1.1 Game 主类

Game 类是游戏入口，负责初始化 Pygame、创建显示窗口、启动游戏主循环。

Listing 5: Game 主循环伪代码

```
1 def main():
```

```
2     pygame.init()
3     screen = pygame.display.set_mode((800, 600))
4     clock = pygame.time.Clock()
5
6     sound_system = SoundSystem()
7     player_stats = PlayerStats()
8     scene_manager = SceneManager(screen, sound_system)
9
10    # 注册场景
11    scene_manager.add_scene('village', VillageScene(screen,
12    player_stats))
13    scene_manager.add_scene('temple', TempleScene(screen,
14    player_stats))
15    scene_manager.add_scene('battle', BattleScene(screen,
16    player_stats))
17    scene_manager.add_scene('settlement', SettlementScene(screen,
18    player_stats))
19
20    scene_manager.set_current_scene('village', use_fade=False)
21
22    while running:
23        events = pygame.event.get()
24        scene_manager.handle_events(events)
25        scene_manager.update()
26        scene_manager.draw()
27        sound_system.update()
28
29        if current_scene.next_scene:
30            scene_manager.set_current_scene(next_scene)
31
32        pygame.display.flip()
33        clock.tick(40)
```

5.1.2 SceneManager 场景管理器

SceneManager 负责场景的注册、切换和渐变过渡。

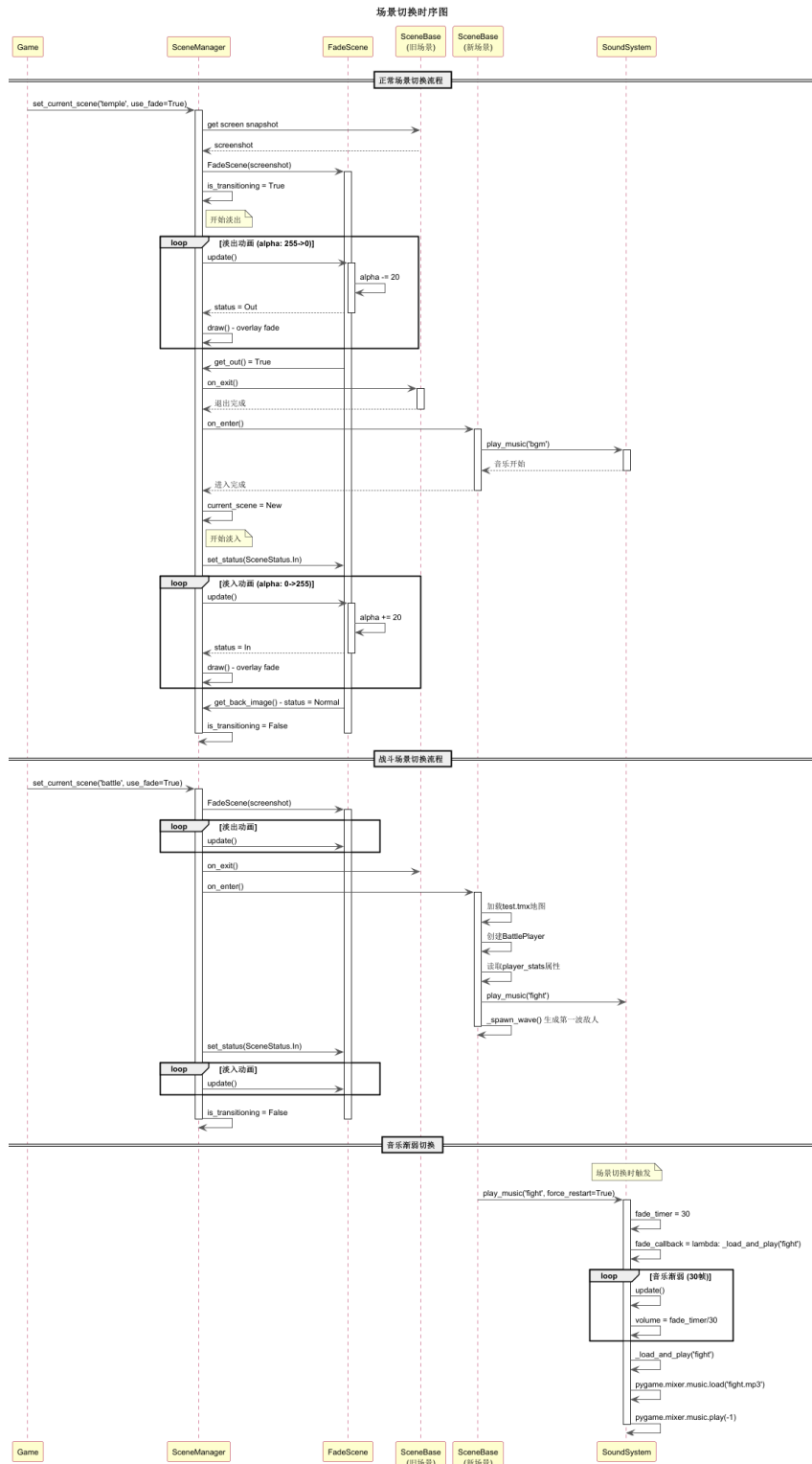


图 5: 场景切换时序图

核心算法:

Listing 6: 场景切换伪代码

```
1 def set_current_scene(name, use_fade=True):
```

```

2     if use_fade and self.current_scene:
3         # 截图当前屏幕
4         screenshot = self.screen.copy()
5         # 创建渐变场景
6         self.fade_scene = FadeScene(screenshot)
7         self.is_transitioning = True
8         self.next_scene_name = name
9     else:
10        # 直接切换
11        if self.current_scene:
12            self.current_scene.on_exit()
13            self.current_scene = self.scenes[name]
14            self.current_scene.on_enter()
15
16    def update(self):
17        if self.is_transitioning:
18            self.fade_scene.update()
19            if self.fade_scene.get_out(): # 淡出完成
20                # 切换到新场景
21                self.current_scene.on_exit()
22                self.current_scene = self.scenes[self.next_scene_name]
23                self.current_scene.on_enter()
24                self.fade_scene.set_status(SceneStatus.In)
25            if self.fade_scene.get_status() == SceneStatus.Normal:
26                self.is_transitioning = False

```

5.2 角色层模块

5.2.1 Player 探索玩家

玩家在村庄和寺庙场景中使用，支持 WASD/方向键移动，具备碰撞检测。

Listing 7: Player 移动算法

```

1    def update(self, keys, obstacles, walkable_areas):
2        if self.is_talking:
3            return # 对话时禁止移动
4
5        dx, dy = 0, 0
6        speed = self.speed // 2 if keys[pygame.K_LSHIFT] else self.speed
7

```

```

8      # 方向键设置方向（用于动画）
9      if keys[pygame.K_DOWN]: self.direction = self.DOWN
10     elif keys[pygame.K_LEFT]: self.direction = self.LEFT
11     elif keys[pygame.K_UP]: self.direction = self.UP
12     elif keys[pygame.K_RIGHT]: self.direction = self.RIGHT
13
14     # 累积移动量（支持斜向移动）
15     if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy += speed
16     if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx -= speed
17     if keys[pygame.K_UP] or keys[pygame.K_w]: dy -= speed
18     if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx += speed
19
20     # 斜向移动归一化
21     if dx != 0 and dy != 0:
22         dx *= 0.707
23         dy *= 0.707
24
25     # 碰撞检测
26     foot_rect = self.get_foot_rect()
27     test_rect = foot_rect.move(dx, dy)
28
29     # 检查障碍物碰撞
30     for obstacle in obstacles:
31         if test_rect.colliderect(obstacle):
32             dx, dy = 0, 0
33             break
34
35     # 更新位置
36     self.pos_x += dx
37     self.pos_y += dy

```

5.2.2 BattlePlayer 战斗玩家

战斗玩家在战斗场景中使用，增加攻击状态和伤害处理。

Listing 8: 攻击范围计算

```

1 def get_attack_rect(self):
2     center_x = self.pos_x + self.width // 2
3     center_y = self.pos_y + self.height // 2

```

```

4
5 if self.direction == self.DOWN:
6     return pygame.Rect(center_x - 40, center_y, 80, 60)
7 elif self.direction == self.UP:
8     return pygame.Rect(center_x - 40, center_y - 60, 80, 60)
9 elif self.direction == self.LEFT:
10    return pygame.Rect(center_x - 60, center_y - 20, 60, 40)
11 elif self.direction == self.RIGHT:
12    return pygame.Rect(center_x, center_y - 20, 60, 40)

```

5.2.3 EnemyBase 敌人 AI

敌人 AI 采用状态机模式，包含站立、追击、攻击三种状态。

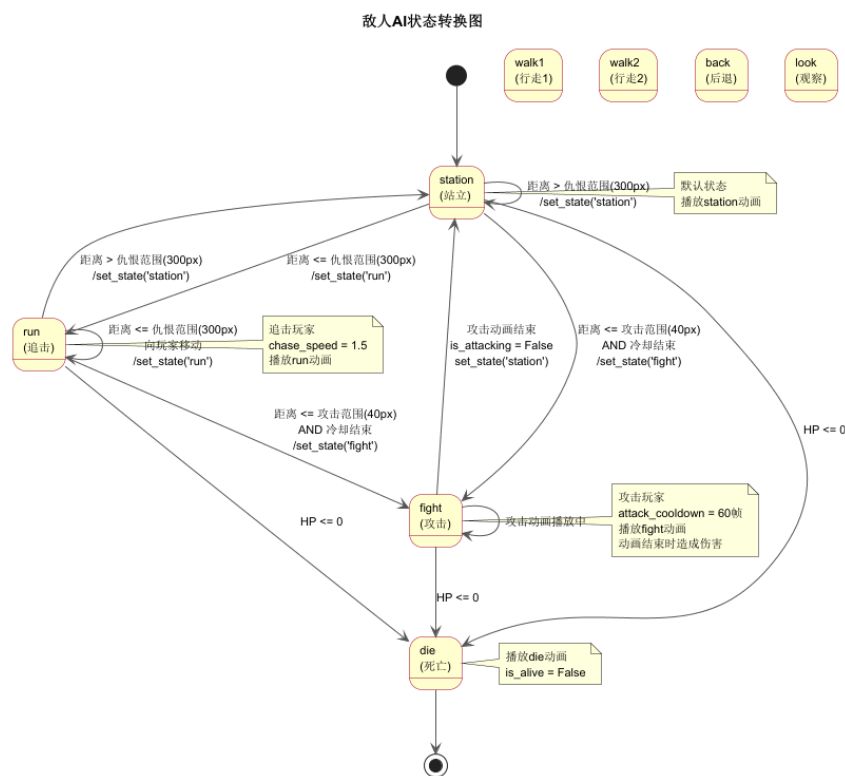


图 6: 敌人 AI 状态机

Listing 9: 敌人追击 AI 算法

```

1 def chase_and_attack(self, player_x, player_y):
2     if not self.is_alive:
3         return
4
5     # 计算与玩家的距离

```



```

6      dx = player_x - self.pos_x
7      dy = player_y - self.pos_y
8      distance = (dx**2 + dy**2) ** 0.5
9
10     # 攻击冷却计时
11     if self.attack_cooldown_timer > 0:
12         self.attack_cooldown_timer -= 1
13
14     # 攻击状态
15     if self.is_attacking:
16         return
17
18     # 攻击范围内且冷却结束
19     if distance <= self.attack_range and self.attack_cooldown_timer
20         <= 0:
21         self.is_attacking = True
22         self.set_state('fight')
23         self.attack_cooldown_timer = self.attack_cooldown
24         return
25
26     # 仇恨范围内追击
27     if distance <= self.aggro_range:
28         self.set_state('run')
29         # 归一化移动方向
30         if distance > 0:
31             dx = dx / distance * self.chase_speed
32             dy = dy / distance * self.chase_speed
33             self.pos_x += dx
34             self.pos_y += dy
35     else:
36         self.set_state('station')

```

5.3 系统层模块

5.3.1 SoundSystem 音效系统

音效系统提供 BGM 播放和渐弱切换功能。

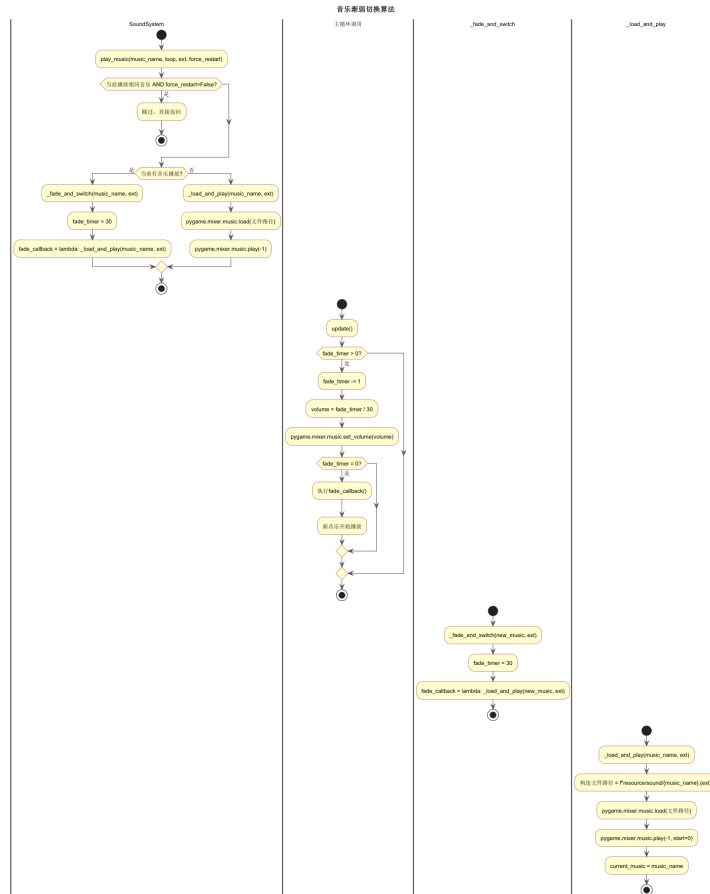


图 7: 音乐渐弱切换算法

Listing 10: 音乐渐弱切换算法

```

1 def play_music(self, music_name, loop=True, ext='mp3', force_restart=
  False):
2     # 检查是否需要切换
3     if music_name == self.current_music and not force_restart:
4         return
5
6     if self.current_music:
7         # 渐弱切换
8         self.fade_timer = self.fade_duration
9         self.fade_callback = lambda: self._load_and_play(music_name,
10             ext)
11     else:
12         # 直接播放
13         self._load_and_play(music_name, ext)
14
15 def update(self):

```

```

15     if self.fade_timer > 0:
16         self.fade_timer -= 1
17         # 线性插值音量
18         volume = self.fade_timer / self.fade_duration
19         pygame.mixer.music.set_volume(volume)
20
21     if self.fade_timer == 0:
22         # 执行切换回调
23         self.fade_callback()

```

5.3.2 DialogSystem 对话系统

Listing 11: 对话系统伪代码

```

1  def start_dialog(self, dialog):
2      self.is_dialog_active = True
3      self.current_dialog = dialog
4      # 创建半透明对话框
5      self.dialog_surface = pygame.Surface(
6          (SCREEN_WIDTH - 100, 120), pygame.SRCALPHA
7      )
8      self.dialog_surface.fill((0, 0, 0, 200))
9
10 def draw(self, screen):
11     if not self.is_dialog_active:
12         return
13     # 绘制对话框背景
14     screen.blit(self.dialog_surface, (50, SCREEN_HEIGHT - 150))
15     # 绘制说话者和内容
16     text = f"{self.current_dialog['speaker']}: {self.current_dialog['text']}"
17     # 绘制操作提示
18     hint = "按空格键继续 ⏏ 按 Esc 键退出"

```

5.4 场景层模块

5.4.1 VillageScene 村庄场景

村庄场景是游戏的中心枢纽，玩家在此与 NPC 交互获取属性加成。

5.4.2 BattleScene 战斗场景

战斗场景实现完整的实时战斗系统，包括波次系统和 HUD 显示。

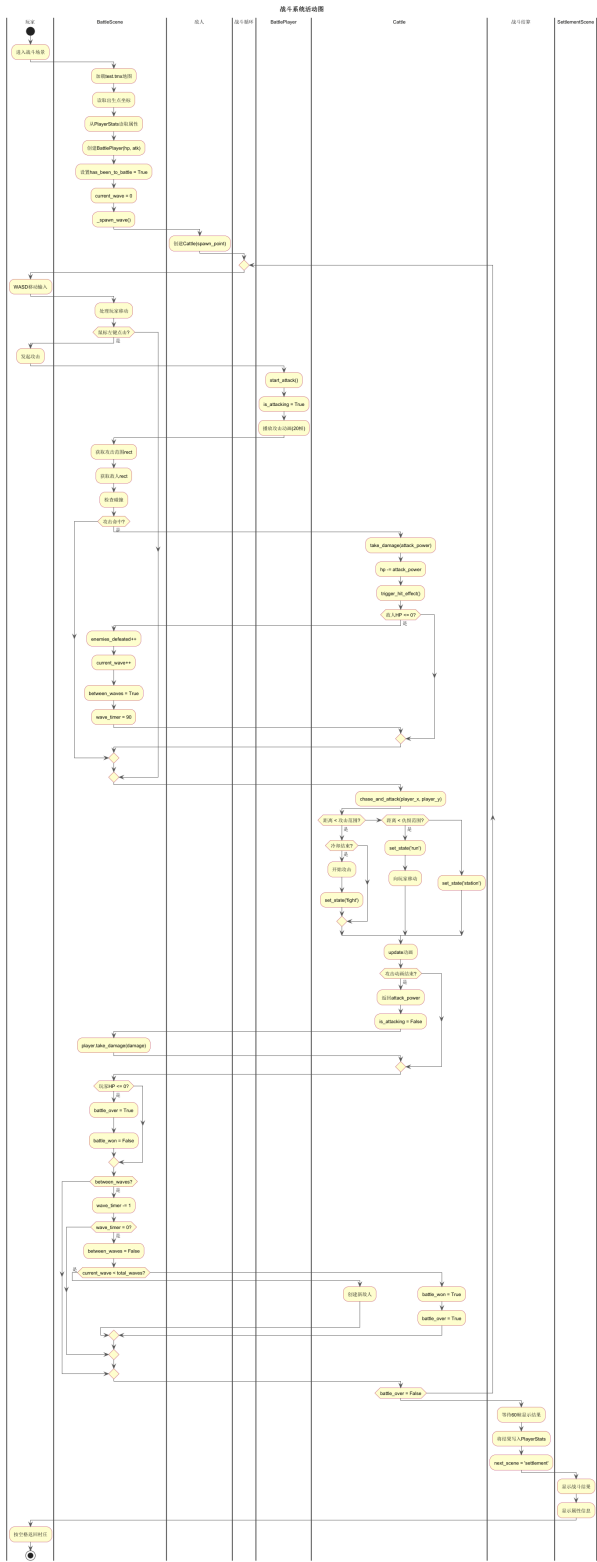


图 9: 战斗系统活动图

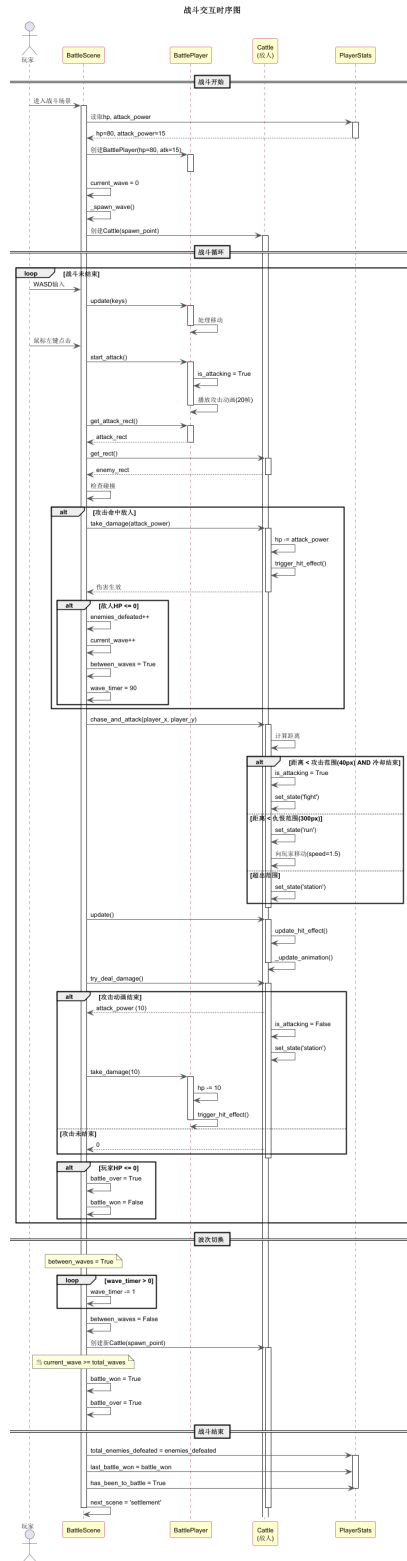


图 10: 战斗交互时序图

核心算法:

Listing 12: 波次切换算法

```
1 def update(self):
2     if self.battle_over:
3         return
4
5     # 更新玩家
6     self.player.update(pygame.key.get_pressed())
7
8     # 更新敌人AI
9     for enemy in self.enemies:
10         if enemy.is_alive:
11             enemy.chase_and_attack(
12                 self.player.pos_x, self.player.pos_y
13             )
14             enemy.update()
15             damage = enemy.try_deal_damage()
16             if damage > 0:
17                 self.player.take_damage(damage)
18
19     # 检测玩家攻击
20     # (在handle_events中处理鼠标点击)
21
22     # 波次切换
23     if self.between_waves:
24         self.wave_timer -= 1
25         if self.wave_timer <= 0:
26             self.between_waves = False
27             if self.current_wave < self.total_waves:
28                 self._spawn_wave()
29             else:
30                 self.battle_won = True
31                 self.battle_over = True
32
33     # 检查玩家死亡
34     if not self.player.is_alive:
35         self.battle_won = False
36         self.battle_over = True
```

5.5 工具层模块

5.5.1 TiledScene TMX 地图渲染器

Listing 13: TMX 地图加载伪代码

```
1 class TiledScene:
2     def __init__(self, tmx_path):
3         self.tmx_data = pytmx.util_pygame.load_pygame(tmx_path)
4         self.map_width = self.tmx_data.width * self.tmx_data.
            tilewidth
5         self.map_height = self.tmx_data.height * self.tmx_data.
            tileheight
6
7     def render_map(self, surface, scroll_x, scroll_y):
8         for layer in self.tmx_data.visible_layers:
9             if isinstance(layer, pytmx.TiledTileLayer):
10                 for x, y, image in layer.tiles():
11                     surface.blit(
12                         image,
13                         (x * self.tmx_data.tilewidth - scroll_x,
14                          y * self.tmx_data.tileheight - scroll_y)
15                     )
16             elif isinstance(layer, pytmx.TiledImageLayer):
17                 surface.blit(layer.image, (0, 0))
```

5.5.2 FadeScene 渐变效果

Listing 14: 渐变效果伪代码

```
1 class FadeScene:
2     alpha_step = 20
3
4     def __init__(self, back_image):
5         self.back_image = back_image
6         self.alpha = 255
7         self.status = SceneStatus.Out
8
9     def update(self):
10         if self.status == SceneStatus.Out:
11             self.alpha -= self.alpha_step
```

```
12         if self.alpha <= 0:
13             self.alpha = 0
14             self.status = SceneStatus.Over
15     elif self.status == SceneStatus.In:
16         self.alpha += self.alpha_step
17         if self.alpha >= 255:
18             self.alpha = 255
19             self.status = SceneStatus.Normal
20
21     def get_back_image(self, x, y):
22         image = self.back_image.copy()
23         image.set_alpha(self.alpha)
24         return image
```

6 算法设计

6.1 碰撞检测算法

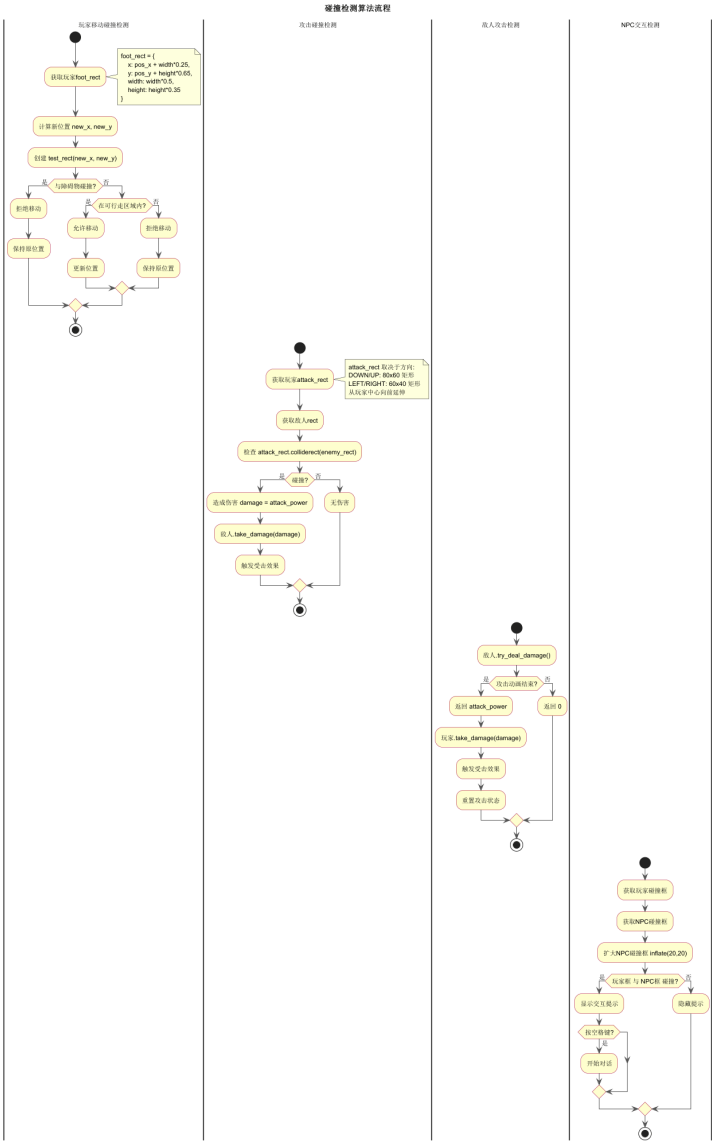


图 11: 碰撞检测算法流程

碰撞检测采用矩形碰撞（AABB）方式：

表 7: 碰撞检测类型

检测类型	碰撞框	说明
玩家-障碍物	foot_rect	脚部区域（50% 宽 × 35% 高）

续下页

续表

检测类型	碰撞框	说明
玩家-NPC	inflate(20,20)	扩大检测范围
玩家-怪物	inflate(40,40)	扩大检测范围
攻击-敌人	attack_rect	方向相关的挥砍范围
敌人-玩家	attack_rect	敌人攻击范围

6.2 波次切换算法

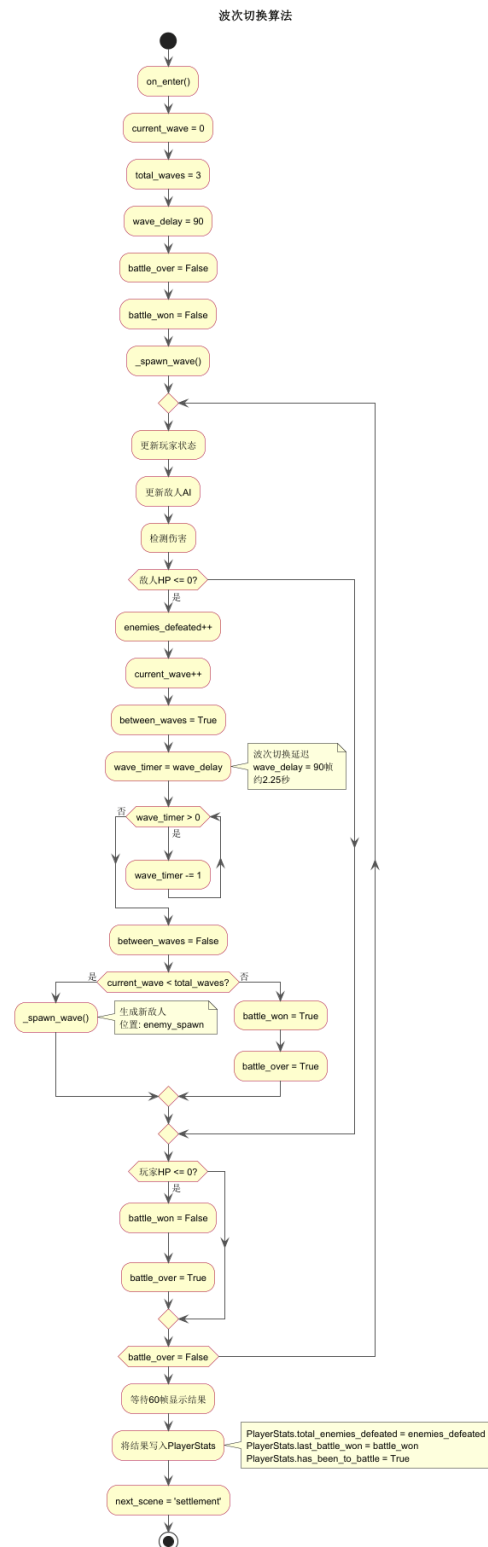


图 12: 波次切换算法流程

波次系统设计:

- 总波次数: TOTAL_WAVES = 3
- 波次间隔: WAVE_DELAY = 90 帧 (约 2.25 秒)
- 每波生成 1 个 Cattle 敌人
- 击败当前波敌人后开始下一波计时
- 全部击败即为战斗胜利

7 运行环境与部署

7.1 运行环境要求

表 8: 运行环境要求

项目	要求
操作系统	Windows 10/11
Python 版本	Python 3.0 及以上
Pygame 版本	Pygame 2.0 及以上
pytmx 版本	pytmx 3.0 及以上
分辨率	800×600 像素
帧率	40 FPS

7.2 依赖库说明

表 9: 依赖库说明

库名称	版本	用途
pygame	2.0+	游戏开发框架，提供窗口、事件、渲染、音频
pytmx	3.0+	TMX 地图文件解析和渲染
python	3.0+	编程语言

7.3 项目部署说明

1. 安装 Python 3.0 及以上版本

-
2. 安装依赖库: `pip install pygame pytmx`
 3. 确保 `resource` 目录下的资源文件完整
 4. 运行主程序: `python main.py`

8 附录

8.1 项目结构

Listing 15: 项目目录结构

```
1 journey_to_the_west/  
2     main.py           # 游戏入口  
3     config.py         # 全局配置  
4     core/  
5         scene_manager.py # 场景管理器  
6     actors/  
7         action.py       # 动画行为类  
8         base_actor.py   # 角色基类  
9         player.py       # 探索玩家  
10        battle_player.py # 战斗玩家  
11        npc.py          # NPC基类  
12        elder.py        # 村民  
13        god.py          # 土地公  
14        tang.py         # 唐僧  
15        enemy.py        # 敌人基类+牛怪  
16    systems/  
17        dialog.py       # 对话系统  
18        player_stats.py # 属性系统  
19        sound.py        # 音效系统  
20        collision.py     # 碰撞系统  
21    scenes/  
22        base_scene.py    # 场景基类  
23        village.py       # 村庄场景  
24        temple.py        # 寺庙场景  
25        battle_scene.py  # 战斗场景  
26        settlement_scene.py # 结算场景  
27        end_scene.py     # 结束场景  
28    utils/
```

```

29         tiled_render.py      # TMX 渲染器
30         fade_scene.py        # 渐变效果
31     resource/                 # 资源文件
32         img/                  # 图片资源
33         sound/                # 音频资源
34         tmx/                  # 地图文件
35         font/                 # 字体文件

```

8.2 数据流图

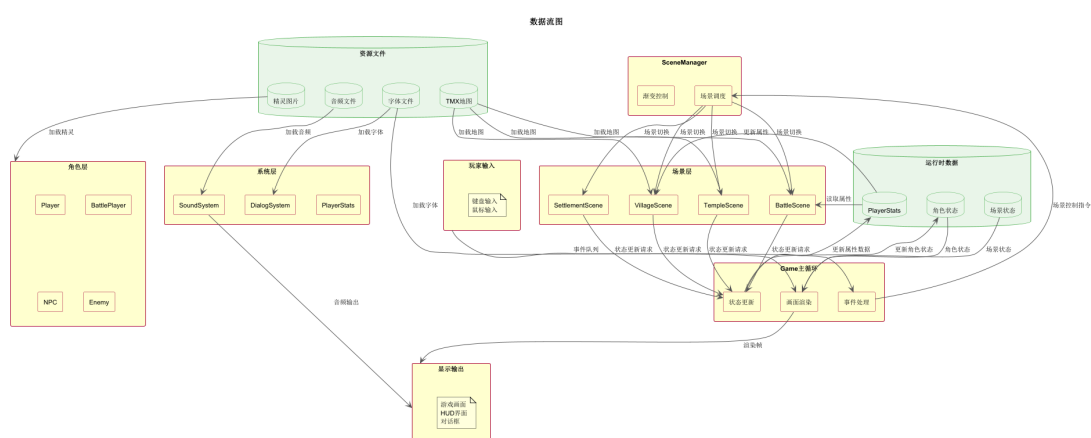


图 13: 数据流图